IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

# Medir un modelo con honestidad

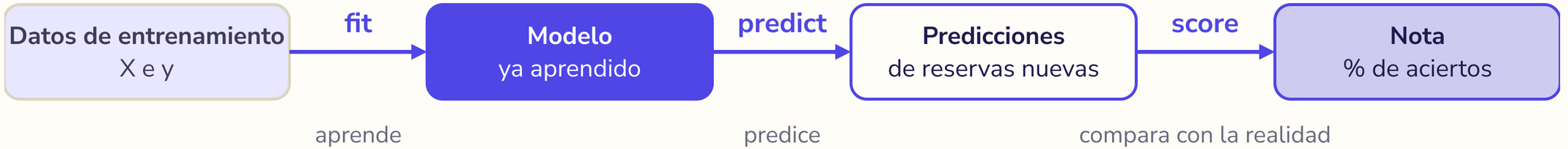
Teoría, práctica y, al final, material de consulta

Sprint 6 · S06_04 · teoría breve, 45 min de práctica y material de consulta

EL PATRÓN DE SCIKIT-LEARN

# Todos los modelos: fit, predict, score

Todos los modelos de scikit-learn se manejan con **los mismos tres verbos**: cambiar de modelo es cambiar una línea.




**fit**

Aprende con **X** (las características) e **y** (la respuesta correcta).

**predict**

Da la **clase** de reservas nuevas: 0 si no cancela, 1 si cancela.

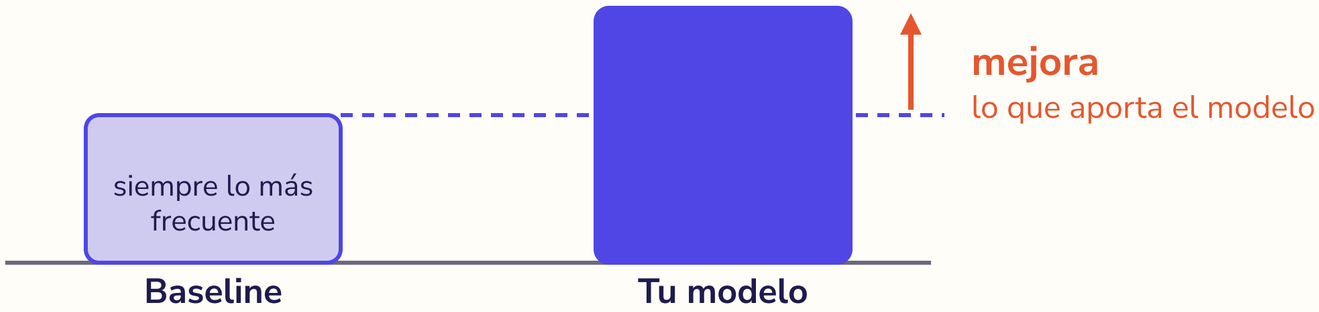
**score**

La **nota** con la métrica por defecto: en clasificación, la exactitud.

EL BASELINE

# Primero, un modelo tonto

El **baseline** predice **siempre la clase más frecuente**, sin mirar los datos. Si tu modelo no lo supera claramente, no aporta nada.




**La cifra que importa no es la nota del modelo, sino su mejora sobre el baseline.**

# Dos aciertos y dos formas de fallar

Para ver **en qué** falla un modelo se cuentan sus aciertos y errores de cada tipo.

| | El modelo dice: no cancela | El modelo dice: cancela |
|---|---|---|
| **Realidad: no cancela** | **VN** · verdadero negativo, acierto | **FP** · falso positivo, falsa alarma |
| **Realidad: cancela** | **FN** · falso negativo, se nos escapa | **VP** · verdadero positivo, acierto |

**Positivo** = lo que buscamos (que cancele). **Verdadero / falso** = si el modelo acertó. Las filas son la realidad y las columnas, la predicción.

MÉTRICAS DE CLASIFICACIÓN

# Cuatro notas, cuatro preguntas

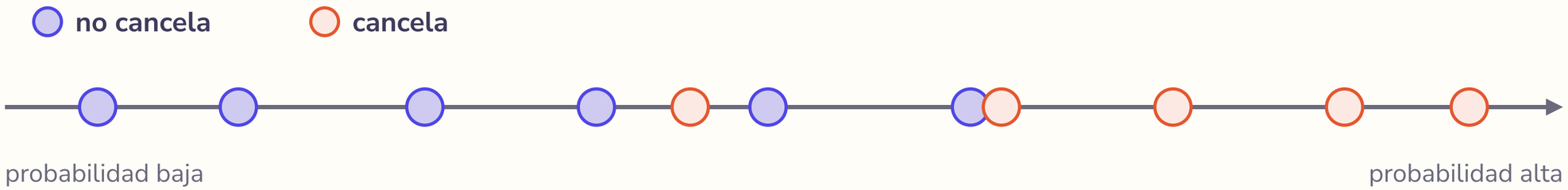
| Métrica | La pregunta | La cuenta |
|---|---|---|
| **Exactitud** | ¿Cuántas acierto en total? | (VN + VP) / todas |
| **Precisión** | De las que aviso «cancela», ¿cuántas cancelan? | VP / (VP + FP) |
| **Recall** | De las que cancelan, ¿cuántas encuentro? | VP / (VP + FN) |
| **F1** | Un solo número que equilibra precisión y recall | media armónica de las dos |

¿Cuál mirar? Si perder una cancelación es caro, el **recall**. Si cada aviso cuesta dinero, la **precisión**.

**La exactitud sola engaña: un baseline que diga siempre «no cancela» acierta mucho y no detecta ninguna cancelación.**

AUC-ROC

# AUC: ¿ordena bien por riesgo?




### Qué mide

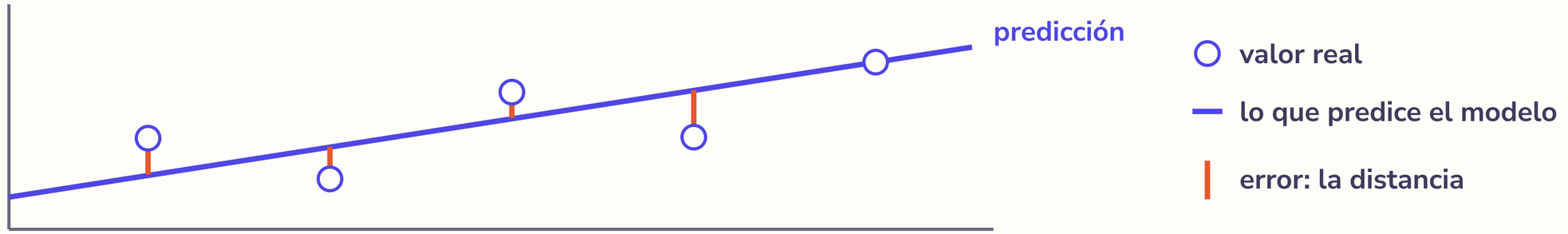
Si el modelo pone **más arriba** a las reservas que cancelan que a las que no, **sin fijar un umbral**.

### Cómo se lee

**1** ordena perfecto · **0,5** como tirar una moneda: es el nivel del baseline.

MÉTRICAS DE REGRESIÓN (CASA)

# Cuando se predice un número




### MAE

Error medio, **en las unidades del dato**: euros, si se predice el precio.

### RMSE

Como el MAE, pero **castiga más** los errores grandes.

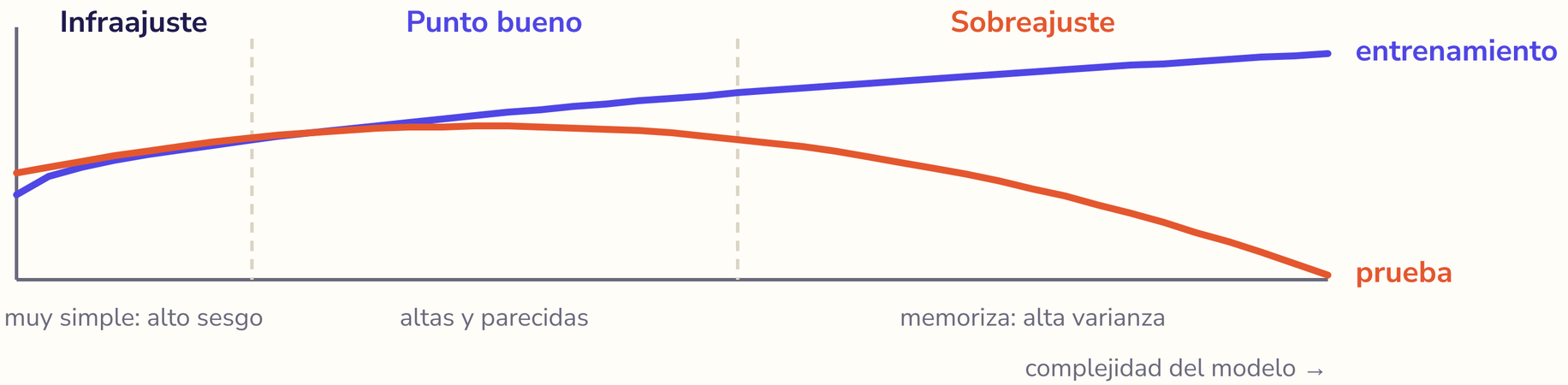
### $R^2$

**1** = perfecto · **0** = no mejora a predecir siempre la media.

SOBREAJUSTE Y SESGO-VARIANZA

# Memorizar no es aprender

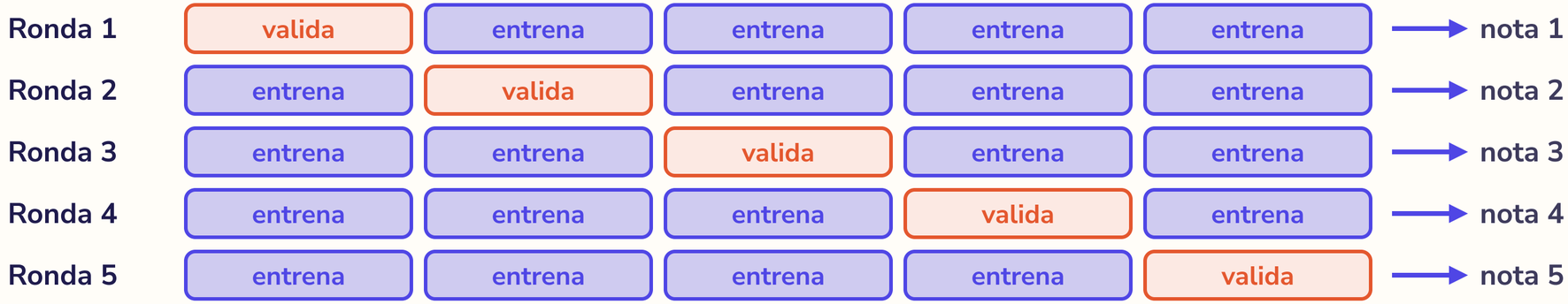
Un modelo muy flexible puede **memorizar** el entrenamiento, como quien se aprende las respuestas del examen de prueba, y fallar con datos nuevos.




**Si el entrenamiento sigue subiendo y la prueba baja, hay sobreajuste.**

VALIDACIÓN CRUZADA

# Cinco exámenes en vez de uno




### Rotando

Se divide el entrenamiento en **5 trozos**: cada ronda valida con uno y entrena con los otros 4.

### Media y desviación

La **media** es tu nota; la **desviación**, cuánto depende de la suerte (pequeña = estable).

FUGA DE DATOS

# La prueba: ¿lo sabía al reservar?

**Fuga de datos** = el modelo usa información que **no tendría en el momento de predecir**. Se decide columna a columna:

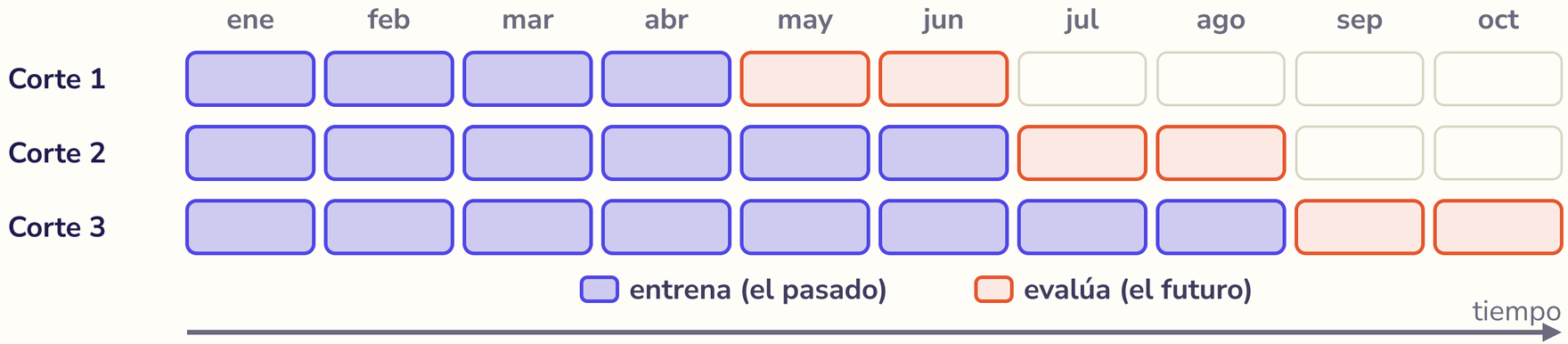
| Columna del hotel | ¿Se conoce al reservar? | Decisión |
|---|---|---|
| **canal** | Sí: se elige al reservar | Se puede usar |
| **noches** | Sí: se indican al reservar | Se puede usar |
| **habitacion_liberada** | No: el hotel la marca al cancelar | **Fuga** |
| **opinion_estancia** | No: se escribe después de alojarse | **Fuga** |
| **hora_checkin_real** | No: solo existe si el cliente llega | **Fuga** |

**Un resultado demasiado bueno es la señal de alarma: antes de celebrarlo, revisa las columnas.**

PARTICIÓN TEMPORAL (CASA)

# Con fechas, valida en el tiempo

El modelo se usará para **predecir el futuro**: hay que evaluarlo como en la vida real, entrenando con el pasado.




**Mezclar al azar es trampa: el modelo ve el futuro al entrenar y la nota sale demasiado optimista.**

PRÁCTICA · CUADERNO S06_04

# Lo que haremos en el cuaderno

| Parte | Qué haremos | Dónde |
|---|---|---|
| **Sección 1** | El baseline: predecir siempre lo mismo (ejemplo) | Clase |
| **Ejercicio 1** | Regresión logística frente al baseline: exactitud y mejora | Clase |
| **Sección 2** | Exactitud, precisión, recall y F1 (ejemplo) | Clase |
| **Ejercicio 2** | Regresión del precio: MAE, RMSE y $R^2$ | Casa |
| **Sección 3 y ejercicio 3** | Curva de sobreajuste y mejor profundidad del árbol | Clase / casa |
| **Ejercicio 4** | AUC con validación cruzada de 5 trozos | Clase |
| **Ejercicio 5** | Detectar las columnas con fuga de datos | Clase |
| **Ejercicio 6** | Validación cruzada temporal | Casa |
| **Secciones 7 y 8** | **GridSearchCV** y mini-reto: tabla de resultados honesta | Casa |

La «Solución de referencia» de cada ejercicio está plegada: se abre solo después de intentarlo.

CONSULTA

# Material de consulta: código y aclaraciones

**Después de intentar el ejercicio** o si algo no te sale. No se proyecta en clase. Mismo **orden que el cuaderno**.

| Ejercicio | Tema | Diapositivas |
|---|---|---|
| **Ejemplo 2 y 1** | Baseline y regresión logística | 14 y 15 |
| **Ejemplo 4** | Matriz de confusión y métricas | 16 y 17 |
| **2 (casa)** | MAE, RMSE y $R^2$ | 18 y 19 |
| **Ejemplo 6 y 3 (casa)** | Curva de sobreajuste y profundidad | 20 y 21 |
| **Ejemplo 8 y 4** | AUC y validación cruzada | 22 a 24 |
| **Ejemplo 10 y 5** | Fuga de datos | 25 y 26 |
| **Ejemplo 12 y 6 (casa)** | Partición temporal | 27 y 28 |
| **Ejemplo 14 (casa)** | Buscar la mejor configuración | 29 |
| **Mini-reto (casa)** | Tabla de resultados honesta | 30 y 31 |

CONSULTA · BASELINE

# Baseline y primer modelo (ejemplo 2 y ejercicio 1)

| 10 reservas nuevas | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| **Real (1 = cancela)** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| **Baseline** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **Modelo** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |

```
modelo = LogisticRegression().fit(X_train, y_train)
modelo.score(X_test, y_test)                                    # 0.9
tonto = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
tonto.score(X_test, y_test)                                     # 0.8
mejora = modelo.score(X_test, y_test) - tonto.score(X_test, y_test)  # 0.1
```

Ejemplo con 8 reservas de entrenamiento y 10 nuevas. En el ejercicio 1 el modelo es **logistica**, un **Pipeline** con **hacer_preprocesado()**.

CONSULTA · ACLARACIONES · BASELINE

# Leer el baseline y el modelo (ejercicio 1)

### Baseline: 0,541

Predice siempre «no se cancela» (solo el 45,9 % se cancela) y acierta en las que **no** se cancelan.

### Modelo: 0,672

**acc_logistica** y una **mejora** de **0,131**: **13 puntos** por encima del baseline.

### Por qué importa **mejora**

La exactitud sola no dice si el modelo vale la pena: la **comparación** con el baseline, sí. Un 0,55 apenas mejoraría 0,541.

### **fit** con train, **score** con test

Test es el examen sorpresa. El pipeline aplica por dentro el preprocesado aprendido en train.

CONSULTA · MÉTRICAS

# Matriz de confusión y métricas (ejemplo 4)

| 100 reservas | Dice: no cancela | Dice: cancela |
|---|---|---|
| **Real: no cancela (80)** | **70** · VN | **10** · FP |
| **Real: cancela (20)** | **5** · FN | **15** · VP |

```
confusion_matrix(y_real, y_pred)  # [[70 10] [ 5 15]]
accuracy_score(y_real, y_pred)    # 0.85 (70 + 15) / 100
precision_score(y_real, y_pred)   # 0.60 15 / (15 + 10)
recall_score(y_real, y_pred)      # 0.75 15 / (15 + 5)
f1_score(y_real, y_pred)          # 0.667 2 · 0,6 · 0,75 / (0,6 + 0,75)
```

El baseline «nunca cancela» acertaría 80 de 100 (exactitud 0,80) y no detectaría ninguna cancelación (recall 0).

CONSULTA · ACLARACIONES · MÉTRICAS

# Leer las cuatro métricas (ejemplo 4)

### Exactitud 0,672

**accuracy_score**: el **67,2 %** de las reservas de test bien clasificadas. Engaña si las clases están desbalanceadas.

### Precisión 0,653

De **513** reservas anunciadas como «se cancelará», **335** se cancelaron: unas 65 de cada 100.

### Recall 0,608

De **551** cancelaciones reales detecta 335: **se le escapan 216**. Unas 61 de cada 100.

### F1 0,630

Resume precisión y recall en un número, entre las dos. ¿Qué error cuesta más? **Depende del negocio.**

CONSULTA · REGRESIÓN (CASA)

# MAE, RMSE y R² a mano (ejercicio 2)

| Precio por noche de 5 reservas (€) | A | B | C | D | E |
|---|---|---|---|---|---|
| **Real** | 90 | 110 | 130 | 100 | 120 |
| **Predicción** | 100 | 105 | 125 | 115 | 120 |
| **Error (sin signo)** | 10 | 5 | 5 | 15 | 0 |

```
mean_absolute_error(real, pred)         # 7.0 (10 + 5 + 5 + 15 + 0) / 5
mean_squared_error(real, pred) ** 0.5   # 8.66 castiga más el error de 15
r2_score(real, pred)                    # 0.625 predecir siempre la media (110 €) daría MAE 12
```

En el ejercicio 2 se entrena una regresión lineal con **tipo_habitacion** y **mes_llegada** y se calculan **mae**, **rmse** y **r2** con los datos de test.

CONSULTA · ACLARACIONES · REGRESIÓN

# Leer las métricas del precio (ejercicio 2, casa)

### MAE ≈ 11,85 €

De media, el modelo se equivoca en unos **12 € por noche**: ≈ 11 % del precio medio en test (unos 104 €).

### RMSE ≈ 14,93 €

Sale **mayor** que el MAE: eleva los errores al cuadrado y castiga los grandes. Siempre es ≥ MAE.

### $R^2$ ≈ 0,80

El tipo de habitación y el mes **explican ≈ el 80 %** de la variación del precio. Un 0 sería predecir siempre la media.

### Dónde se calcula

Con los datos de **test** (**yp_te** y **pred_p**) y con su propia partición **Xp_tr**, **Xp_te**, **yp_tr**, **yp_te**.

CONSULTA · SOBREAJUSTE (CASA)

# La profundidad del árbol (ejemplo 6 y ejercicio 3)

```
for p in range(1, 21):
    arbol = DecisionTreeClassifier(max_depth=p, random_state=SEMILLA).fit(Xtr_p, y_train)
    acc_tr.append(arbol.score(Xtr_p, y_train))                    # entrenamiento: 0,626 → 0,993
    acc_te.append(arbol.score(Xte_p, y_test))                     # prueba: sube hasta 0,657 y baja a ≈ 0,60
mejor_profundidad = list(profundidades)[int(np.argmax(acc_te))]  # 4
brecha_maxima = acc_tr[-1] - acc_te[-1]                           # 0.40
```

### max_depth

Cuántas preguntas de sí o no encadena el árbol: de 1 a 20.

### Dos listas

**acc_tr** y **acc_te** guardan el acierto de cada profundidad, para dibujar las dos curvas.

CONSULTA · ACLARACIONES · SOBREAJUSTE

# Leer las curvas (ejemplo 6 y ejercicio 3, casa)

### Entrenamiento sube sin parar

De **0,626** con profundidad 1 a **0,993** con 20: el árbol acaba **memorizando**.

### Prueba mejora y baja

Mejora hasta la profundidad **4** (≈ 0,657) y después baja hasta ≈ 0,60: **sobreajuste**.

### Sumar 1 a **argmax**

**np.argmax(acc_te)** da la posición **3** (se cuenta desde 0) y las profundidades empiezan en 1: 3 ↔ profundidad **4**.

### Brecha de 0,40

Con profundidad 20, **0,993** en train y solo **0,597** en test: apenas mejora el baseline (0,541).

CONSULTA · AUC

# Calcular el AUC (ejercicio 4)

| Reserva | A | B | C | D | E |
|---|---|---|---|---|---|
| **Probabilidad de cancelar** | 0,90 | 0,20 | 0,50 | 0,45 | 0,30 |
| **Real** | cancela | no | no | cancela | no |

2 que cancelan × 3 que no = **6 parejas**. ¿La que cancela tiene **más probabilidad**? Sí en 5 de 6 (solo falla D, 0,45, contra C, 0,50).

```
y_real = [1, 0, 0, 1, 0]
proba  = [0.9, 0.2, 0.5, 0.45, 0.3]
roc_auc_score(y_real, proba)  # 0.833
```

**0,83**

**AUC = 5 / 6**

1 = ordena perfecto · 0,5 = como tirar una moneda

CONSULTA · VALIDACIÓN CRUZADA

# Validación cruzada (ejemplo 8 y ejercicio 4)

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEMILLA)
puntuaciones = cross_val_score(logistica, X_train, y_train, cv=cv, scoring="accuracy")
# exactitud por trozo: 0.678  0.675  0.699  0.676  0.712
auc_cv = cross_val_score(logistica, X_train, y_train, cv=cv, scoring="roc_auc")
# AUC por trozo: 0.741  0.744  0.758  0.754  0.776   ·   auc_medio = auc_cv.mean() → 0.755
```

### StratifiedKFold

Mantiene la **proporción de cancelaciones** en cada trozo; **shuffle=True** mezcla las filas antes de repartir.

### cross_val_score

Devuelve **una nota por trozo**: 5 números. Con **scoring="roc_auc"**, el AUC de cada uno.

CONSULTA · ACLARACIONES · VALIDACIÓN CRUZADA

# Leer la validación cruzada (ejemplo 8 y ejercicio 4)

### Exactitud: 0,688 ± 0,015

Oscila entre **0,675 y 0,712**. La media estima el rendimiento (≈ 69 %); la desviación, la **estabilidad** (±1,5 puntos).

### AUC medio ≈ 0,755

**5** puntuaciones, de 0,741 a 0,776. 0,5 es azar; 0,755 = en ≈ **75 %** de las parejas, la cancelada tiene más probabilidad.

### 0,688 ≠ 0,672

La media de la validación cruzada y la exactitud en test se calculan con **datos distintos**: la diferencia (1,6 puntos) es del tamaño de la desviación.

### Sobre **X_train**, no **X_test**

Para decidir y comparar sin gastar la prueba, que se usa **una sola vez** al final.

CONSULTA · FUGA DE DATOS

# Una columna que delata el resultado (ejemplo 10)

```
X_fuga = X.copy()
X_fuga["reembolso_emitido"] = np.where(y == 1, rng.choice([1, 1, 1, 0], len(y)), 0)  # casi = cancelada
auc_real = cross_val_score(logistica, X, y, cv=cv, scoring="roc_auc").mean()         # 0.753
auc_fuga = cross_val_score(modelo_fuga, X_fuga, y, cv=cv, scoring="roc_auc").mean()  # 0.937
```

### AUC sin la columna: 0,753

Un modelo honesto, con lo que se sabe al reservar.

### AUC con la columna: 0,937

¡Demasiado bueno! **reembolso_emitido** es **casi la respuesta**: el modelo ya no predice, lee la solución.

CONSULTA · ACLARACIONES · FUGA DE DATOS

# Detectar las columnas con fuga (ejercicio 5)

### Fuga: 2 columnas

**importe_devuelto** y **fecha_cancelacion**: solo existen **después** de que el cliente cancele (la fecha ni existe si no canceló).

### No son fuga

**tarifa**, **antelacion_dias** y **cancelaciones_previas** se conocen **al reservar**: la tarifa se elige entonces y el historial es anterior.

### La pregunta

«¿Existiría este dato **en el momento de predecir**?» Si la respuesta es no, hay que quitarla del modelo.

### La respuesta

**columnas_con_fuga = ["importe_devuelto", "fecha_cancelacion"]**, en cualquier orden.

CONSULTA · PARTICIÓN TEMPORAL (CASA)

# Entrenar con el pasado (ejemplo 12 y ejercicio 6)

```
cortes = TimeSeriesSplit(n_splits=3)
for pasado, futuro in cortes.split(X):
    print(pasado, futuro)  # [0 1 2 3] [4 5] · [0 1 2 3 4 5] [6 7] · [0 1 2 3 4 5 6 7] [8 9]
```

Ejemplo con 10 reservas, una por mes y **ordenadas por fecha**: siempre se entrena con el pasado y se evalúa con lo más reciente.

```
corte = int(len(df_ord) * 0.8)                # ejemplo 12: el 80 % más antiguo entrena
mod_t.fit(Xo.iloc[:corte], yo.iloc[:corte])
# ejercicio 6: un AUC por cada uno de los 4 cortes
auc_temporal = cross_val_score(mod_t, Xo, yo, cv=TimeSeriesSplit(n_splits=4), scoring="roc_auc")
```

CONSULTA · ACLARACIONES · PARTICIÓN TEMPORAL

# Leer la partición temporal (ejemplo 12 y ejercicio 6)

### Corte: 6 de agosto de 2025

Entrenan las **4.800** reservas más antiguas (80 %) y se evalúa con las **1.200** más recientes.

### Exactitud 0,655

Unos 1,7 puntos menos que la aleatoria (0,672): predecir el futuro es más difícil y esta cifra es **más honesta**.

### Ordenar antes de **iloc[:corte]**

Coge las **primeras filas**: si no estuvieran ordenadas por fecha, el corte mezclaría pasado y futuro.

### **auc_temporal**: 4 cortes

**[0.745 0.75 0.756 0.732]**: estable en el tiempo, aunque el último corte es el más bajo.

CONSULTA · AMPLIACIÓN · GRIDSEARCHCV (CASA)

# Probar varios valores de C (ejemplo 14)

```
rejilla = {"clf__C": [0.01, 0.1, 1, 10]}     # los valores que probar
busqueda = GridSearchCV(logistica, rejilla, cv=cv, scoring="roc_auc").fit(X_train, y_train)
busqueda.best_params_, busqueda.best_score_  # {'clf__C': 0.1} 0.755
```

### Hiperparámetro

Un ajuste que decidimos **nosotros**: no lo aprende el modelo, como la **C** de la regresión logística.

### GridSearchCV

Prueba todas las combinaciones con **validación cruzada** y se queda con la mejor. Solo con **X_train**.

Mejor **C = 0.1**, AUC **0,755**: con **C = 1** (por defecto) sería 0,7548. **Casi nada**: no siempre merece la pena ajustar.

CONSULTA · MINI-RETO (CASA)

# Tabla de resultados honesta

```
base = Pipeline([("prep", hacer_preprocesado()), ("clf", DummyClassifier(strategy="prior"))])
punt = {"baseline": cross_val_score(base, X_train, y_train, cv=cv, scoring="roc_auc"),
        "logistica": cross_val_score(logistica, X_train, y_train, cv=cv, scoring="roc_auc")}
resultados = pd.DataFrame({m: {"auc_cv_media": p.mean(), "auc_cv_desv": p.std()} for m, p in punt.items()}).T.round(3)
```

| modelo | auc_cv_media | auc_cv_desv |
|---|---|---|
| **baseline** | 0,500 | 0,000 |
| **logistica** | 0,755 | 0,012 |

Es la tabla del cuaderno, con los números reales. Falta la frase **conclusion**: ¿compensa el modelo frente al baseline?

CONSULTA · ACLARACIONES · MINI-RETO

# Leer la tabla de resultados (mini-reto, casa)

### Baseline: 0,500 ± 0,000

Da a todas las reservas **la misma probabilidad**: no ordena ninguna por encima de otra y su AUC es siempre 0,5, en los 5 trozos.

### Logística: 0,755 ± 0,012

**25 puntos** sobre el azar y estable entre trozos: **compensa**.

### Una comparación honesta

Hecha con validación cruzada sobre **train** y con un baseline de referencia.

### La conclusión

Una frase con una cifra y un veredicto. Si no ganara con claridad al baseline, dirías que **no compensa**.